Computer Vision and Deep Learning

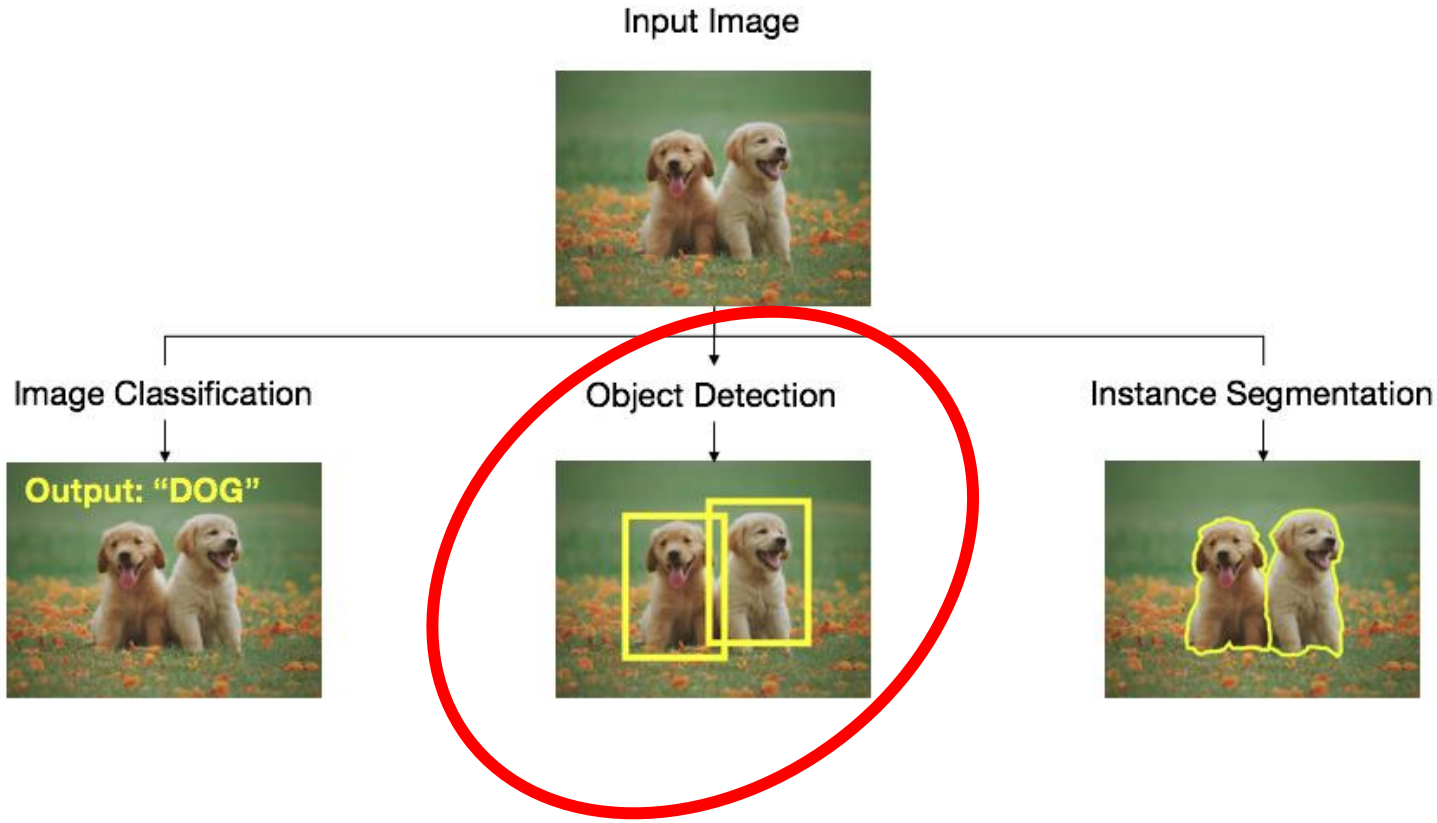
Lab 03 & 04: Object detection

Today & Next Lab

- Revision of the code from the lab02
- Recap of the object detection task
- Pedestrian detection using Faster R-CNN
- Object detection using YOLO (Next LAB Session)

Object detection

- ***Object detection*** is another core task in CV



The problem

- *Image classification* involves assigning a class label to an image
- *Object localization* involves drawing a bounding box around one or more objects in an image
- ***Object detection*** combines these two tasks and draws a bounding box around each object of interest in the image and assigns them a class label

The problem (cont'd)

- Also, you might not necessarily draw just one bounding box since there could be many bounding boxes representing different objects of interest within the image, and you would not know how many beforehand

- This is the major reason why you cannot proceed with this problem by building a "standard" CNN followed by a fully connected layer; the length of the output layer is variable (*i.e.*, not constant) because the number of occurrences of the objects of interest is not fixed

R-CNN

- To bypass the problem of selecting a huge number of regions, Ross Girshick *et al.* proposed ***R-CNN*** [1], a method where we use a selective search algorithm to extract just 2000 regions from the image, called them ***region proposals***

- Therefore, instead of trying to classify a huge number of regions, we can just work with 2000 regions

[1] https://arxiv.org/pdf/1311.2524.pdf

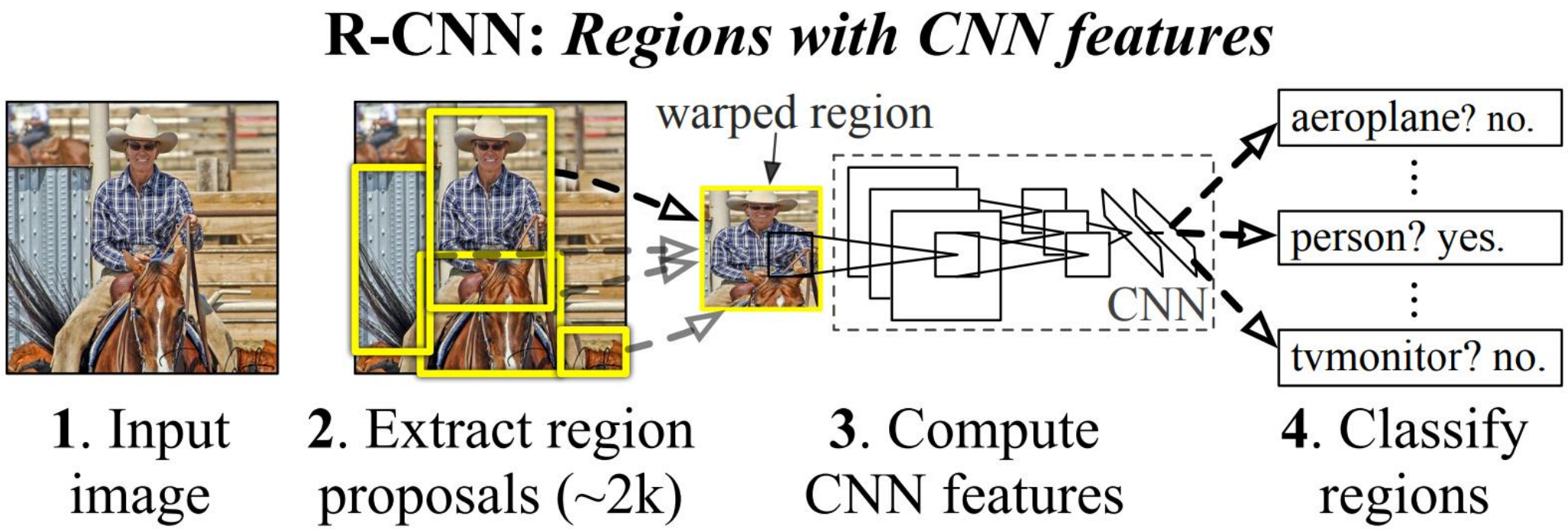
R-CNN (cont'd)

- These 2000 region proposals are generated using the selective search algorithm, which executes the following steps
  - Generate initial sub-segmentation, *i.e.*, we generate many candidate regions
  - Use a greedy algorithm to recursively combine similar regions into larger ones
  - Use the generated regions to produce the final candidate region proposals

- To know more about the selective search algorithm, see [2]

[2] http://www.huppelen.nl/publications/selectiveSearchDraft.pdf

R-CNN (cont'd)


R-CNN: Regions with CNN features
warped region
CNN
aeroplane? no.
person? yes.
tvmonitor? no.
1. Input image
2. Extract region proposals (~2k)
3. Compute CNN features
4. Classify regions

Problems with R-CNN

- It still takes a huge amount of time to train the network as you would have to classify 2000 region proposals per image

- It cannot be implemented real-time as it takes more than 30 seconds for each test image

- The selective search algorithm is a fixed algorithm. Therefore, no learning is happening at that stage. This could lead to the generation of bad candidate region proposals

Fast R-CNN

- The same author of the previous paper (R-CNN) solved some of the drawbacks of R-CNN to build a faster object detection algorithm, and it was called ***Fast R-CNN*** [3]

- With Fast R-CNN, instead of feeding the region proposals to the CNN, we feed the input image to the CNN to generate a convolutional feature map

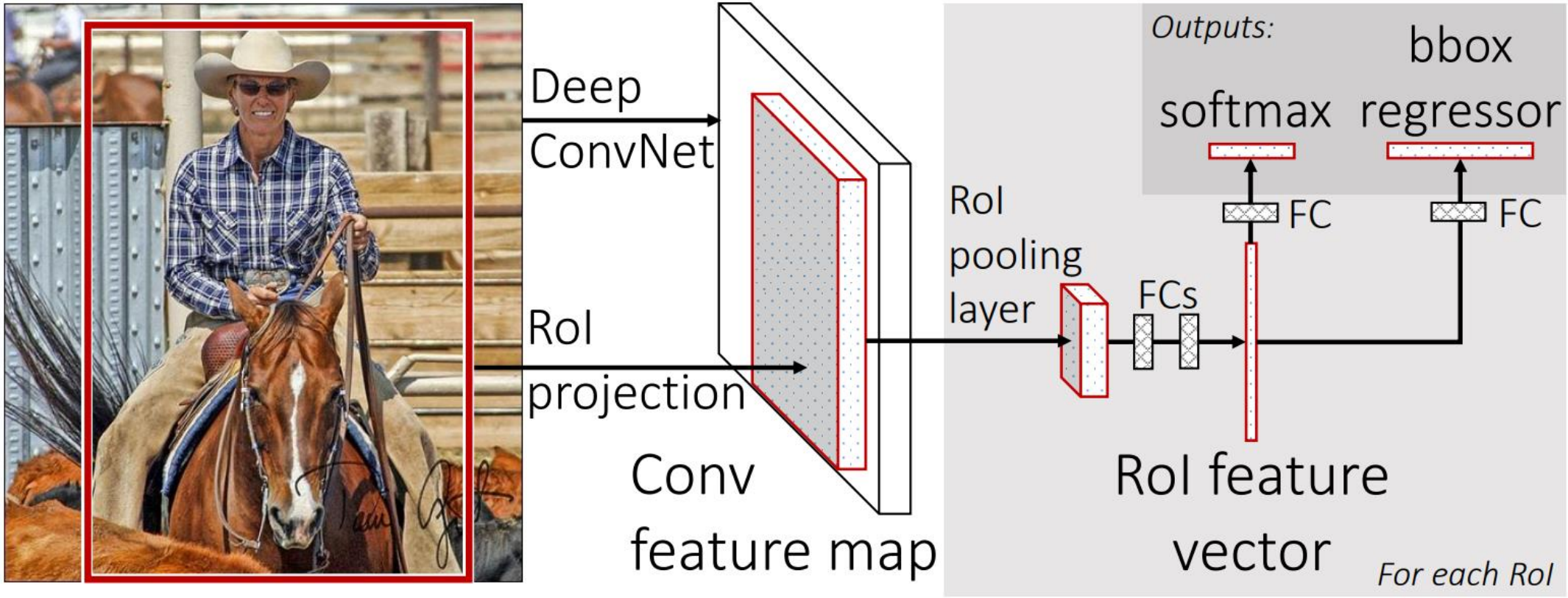
[3] https://arxiv.org/pdf/1504.08083.pdf

Fast R-CNN (cont'd)

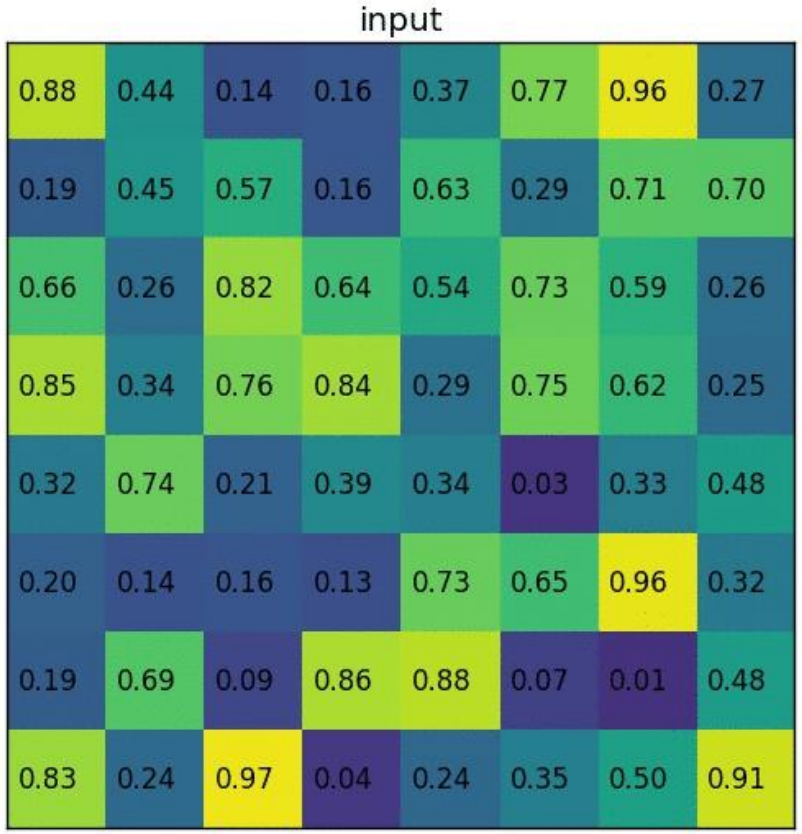
- From the convolutional feature map, we identify the region of proposals and warp them into squares, and by using a ***RoI*** (***R***egion ***o***f ***I***nterest) pooling layer we reshape them into a fixed size so that they can be fed into a fully connected layer

- From the RoI feature vector, we use a softmax layer to predict the class of the proposed region and using regression we predict the offset values for the bounding boxes

Fast R-CNN (cont'd)



Fast R-CNN : ROI Pooling (cont'd)



Faster R-CNN

- Both of the above algorithms (R-CNN & Fast R-CNN) uses selective search to find out the region proposals

- Selective search is a slow and time-consuming process affecting the performance of the network

- Therefore, Shaoqing Ren et al. came up with an object detection algorithm that eliminates the selective search algorithm and lets the network learn the region proposals, creating the ***Faster R-CNN*** [4]

[4] https://arxiv.org/pdf/1506.01497.pdf

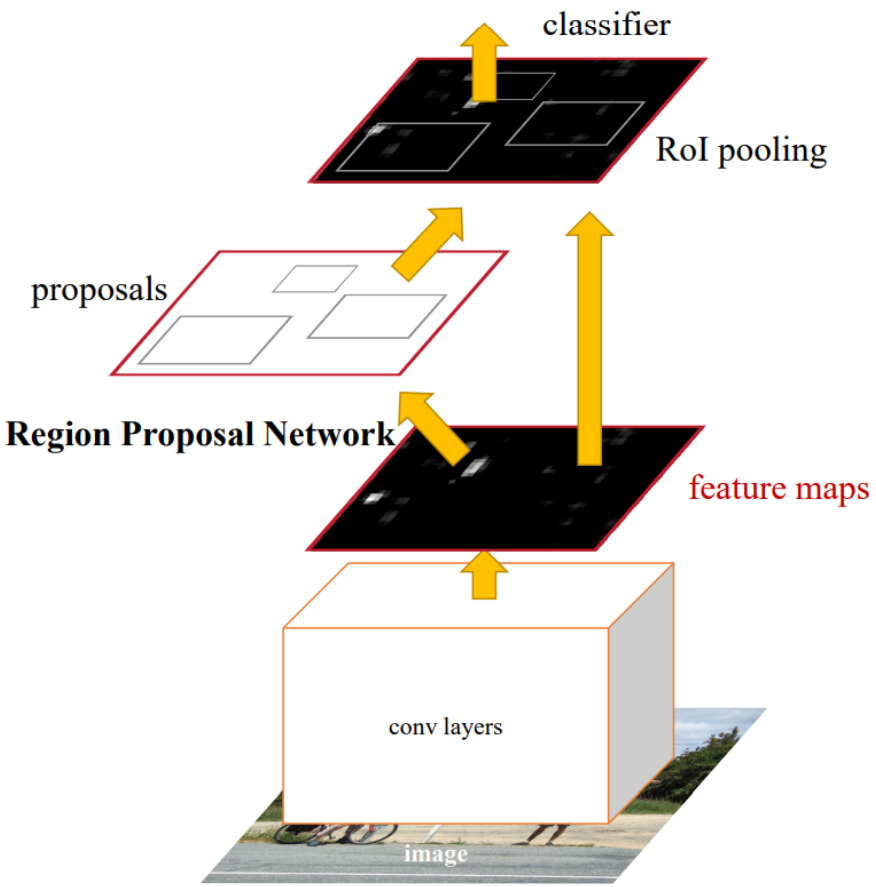
Faster R-CNN (cont'd)

- Similar to Fast R-CNN, the image is provided as an input to a convolutional network which provides a convolutional feature map

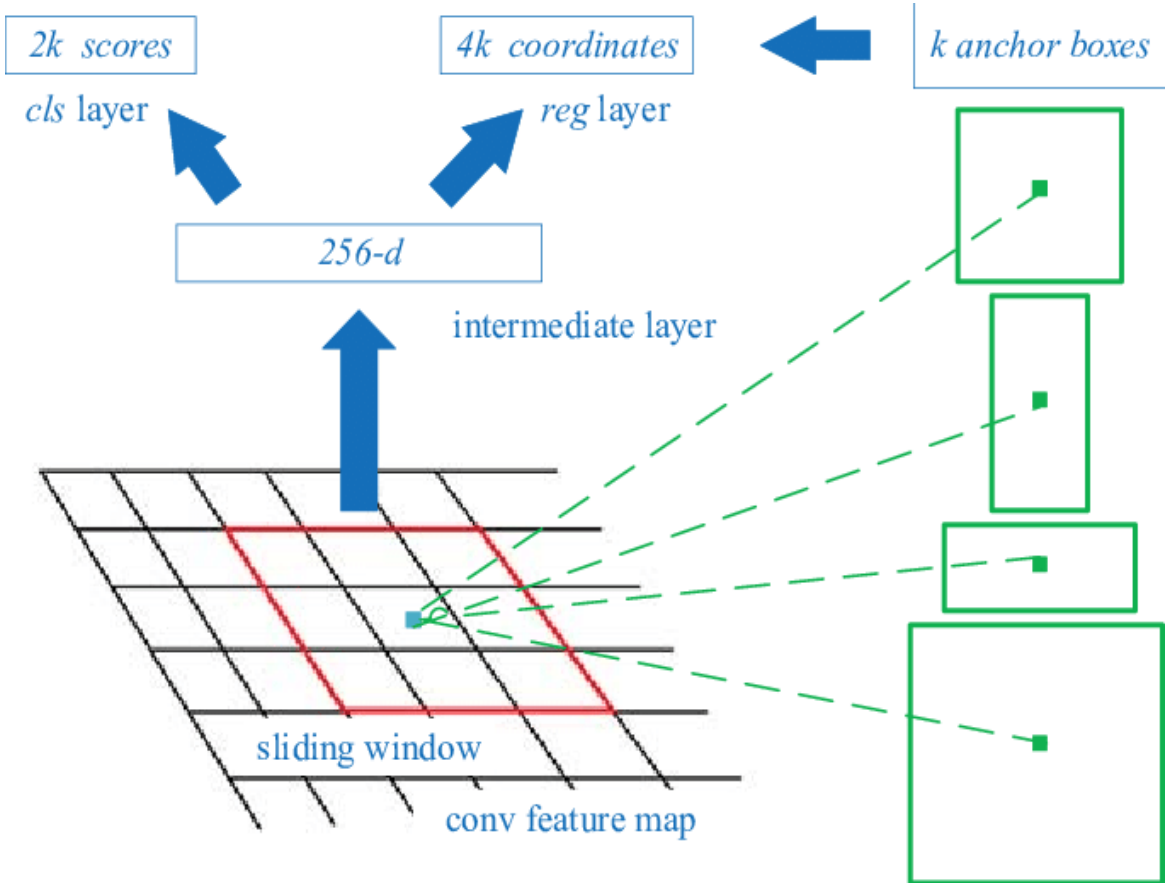
- Instead of using a selective search algorithm on the image to identify the region proposals, a separate network is used to predict the region proposals

- The predicted region proposals are then reshaped using a RoI pooling layer which is then used to classify the image within the proposed region and using regression we predict the offset values for the bounding boxes

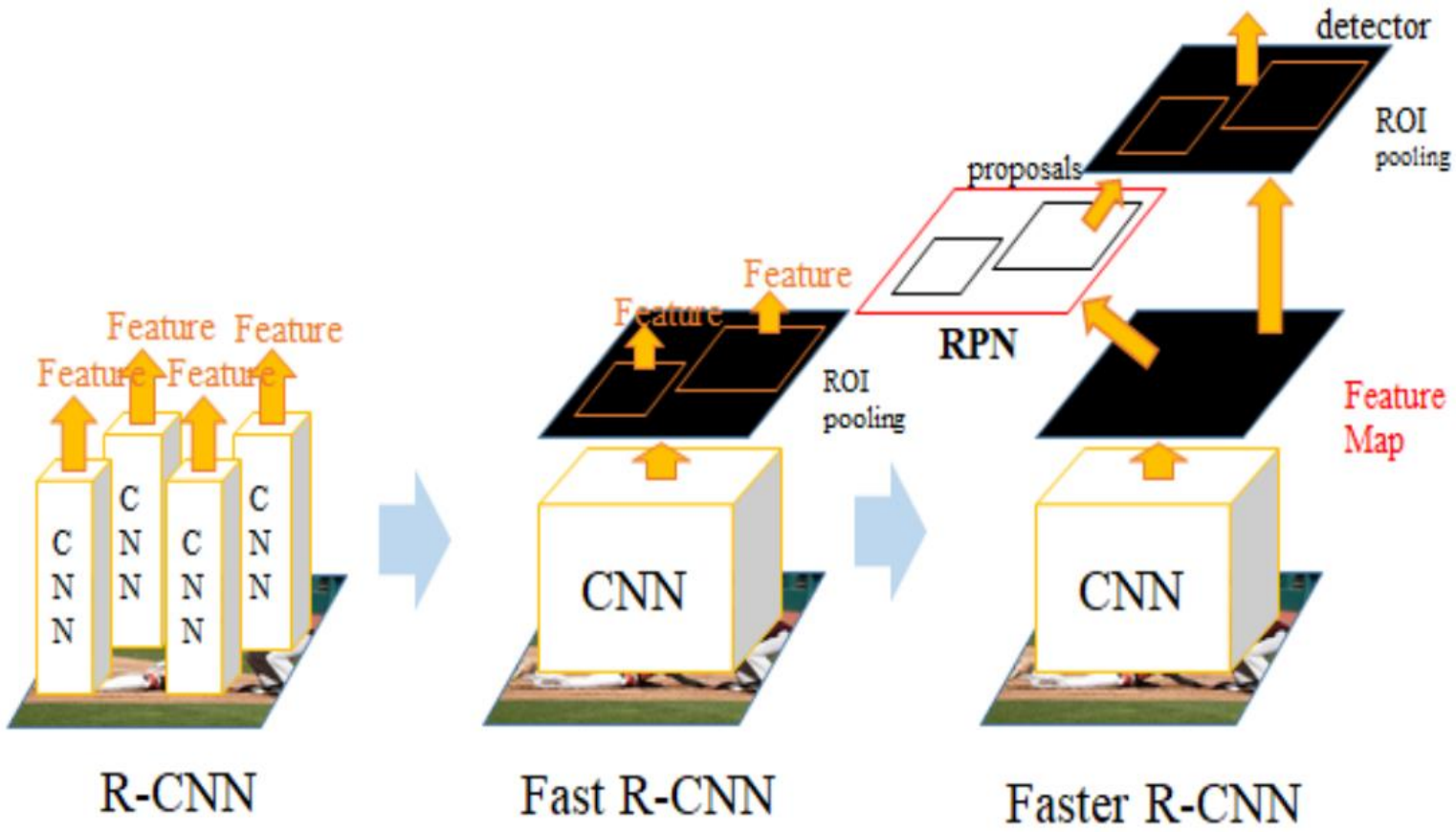
Faster R-CNN (cont'd)



Faster R-CNN : RPN (cont'd)



R-CNN, Fast R-CNN, Faster R-CNN



(*) https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9277040

Exercises

YOLO – You Only Look Once

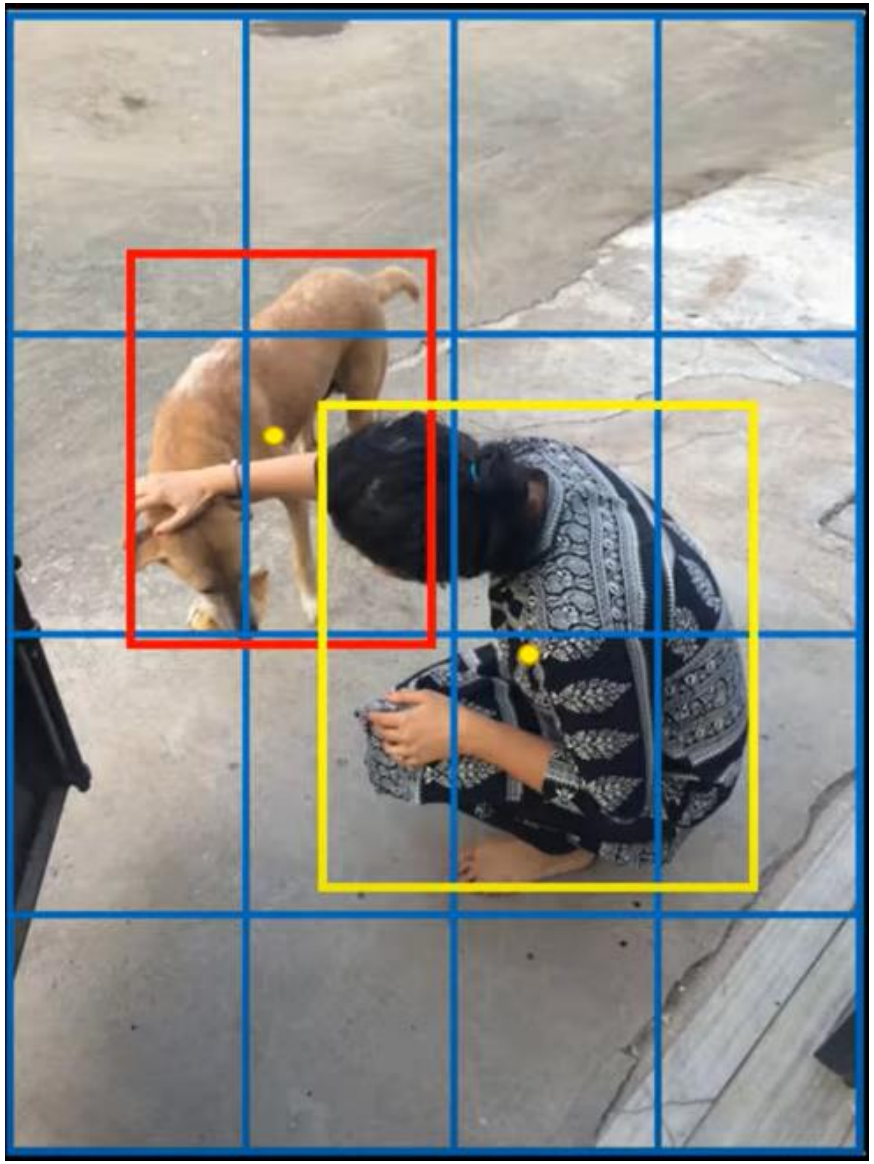
- All of the previous object detection algorithms use regions to localize the object within the image

- ***YOLO*** or ***Y***ou ***O***nly ***L***ook ***O***nce [5] is an object detection algorithm much different from the region-based algorithms seen before. In YOLO, a single convolutional network predicts the bounding boxes and the class probabilities for these boxes

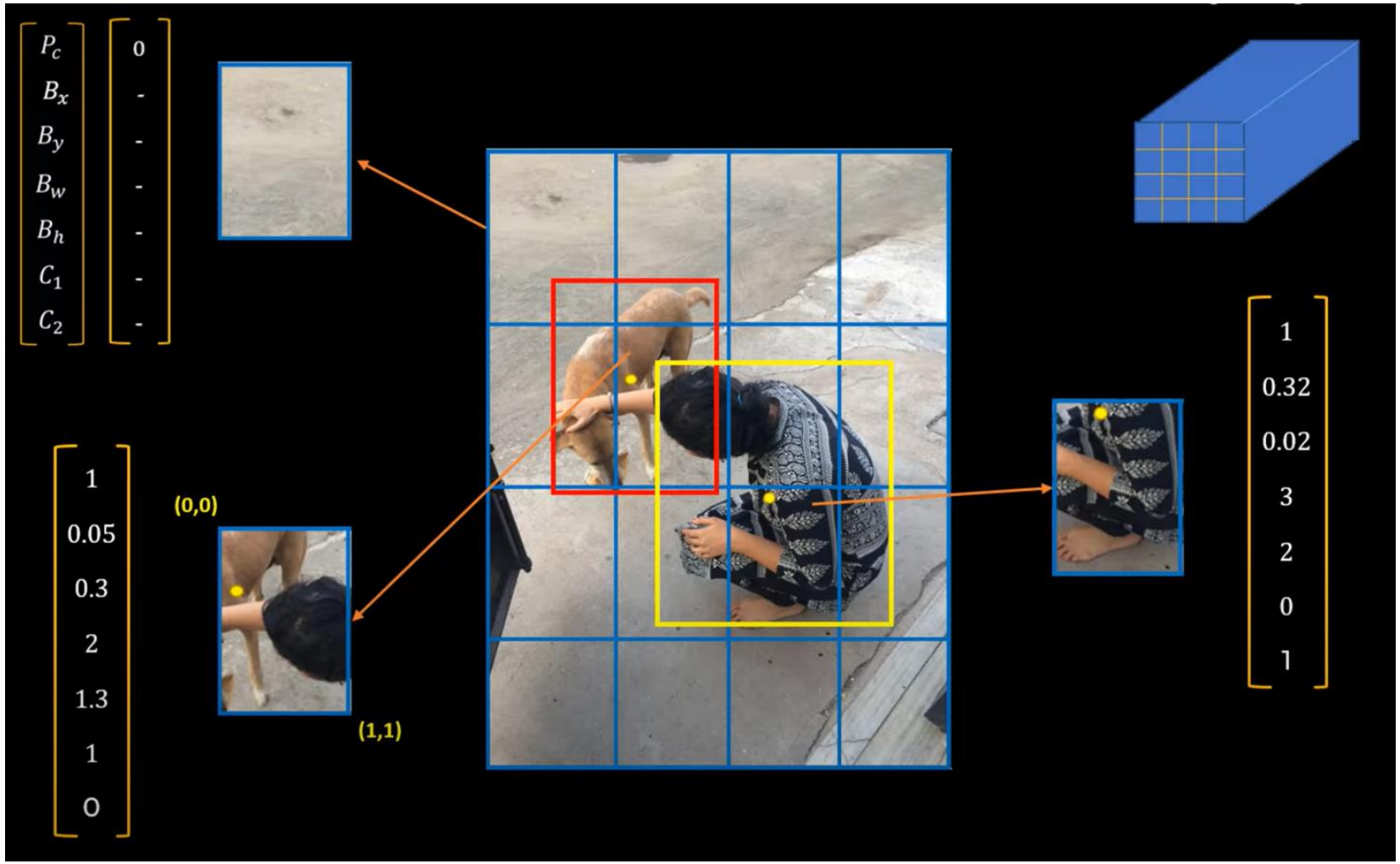
[5] https://arxiv.org/pdf/1506.02640.pdf

YOLO (cont'd)

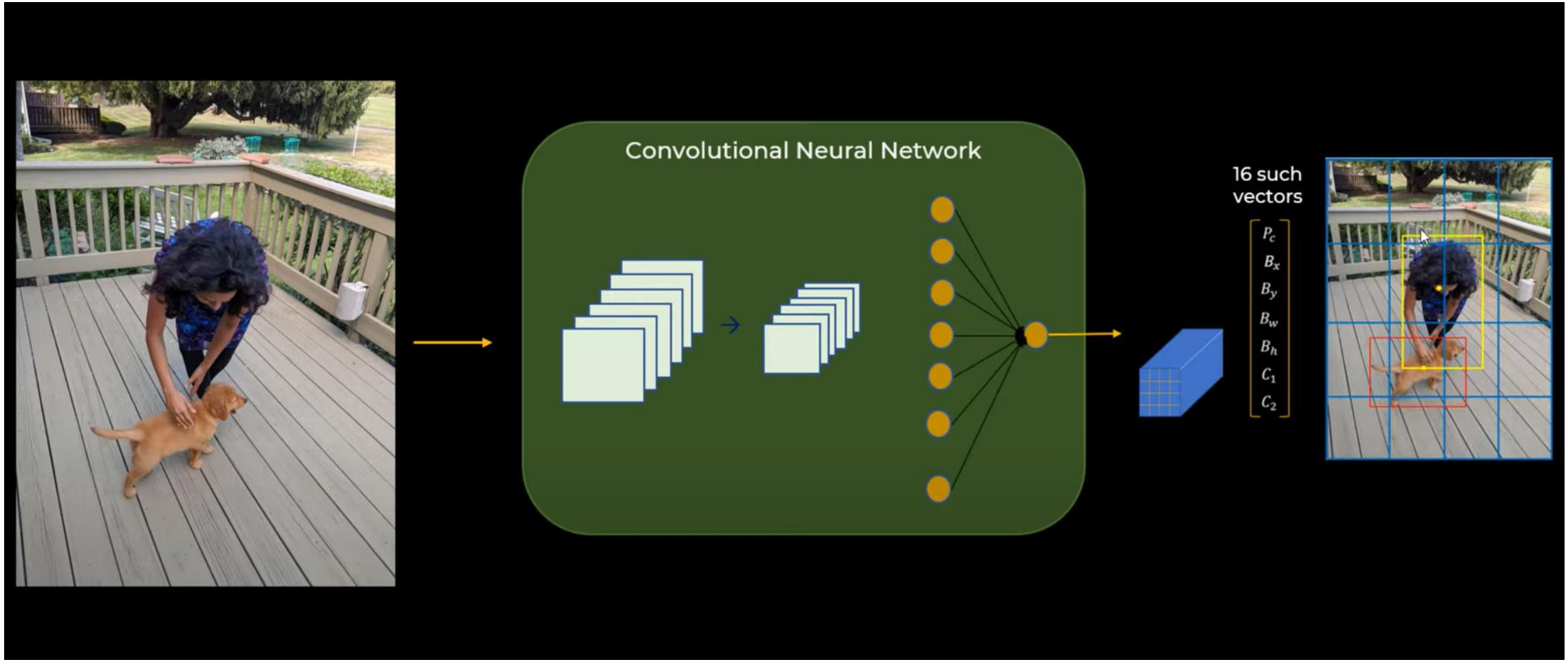
- Let's suppose that we need to find objects in this image
- How YOLO works is that we take an image and split it into an SxS grid



YOLO (cont'd)

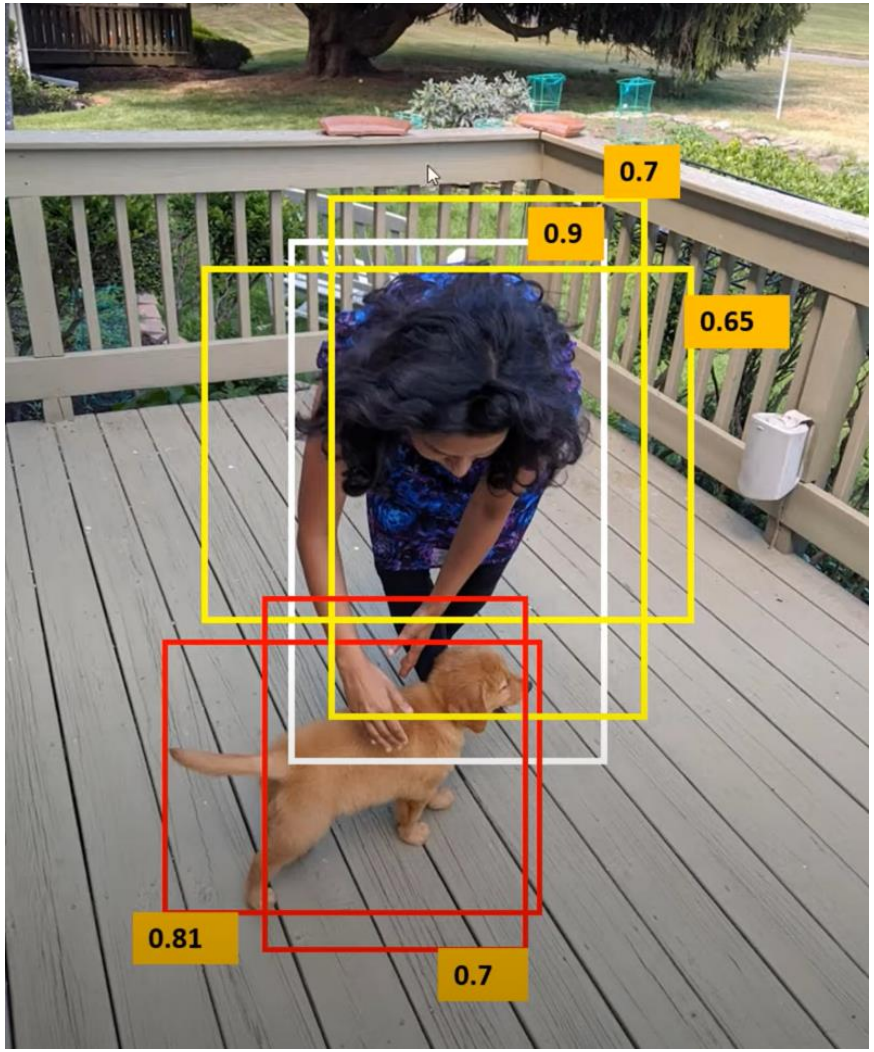


YOLO (cont'd)

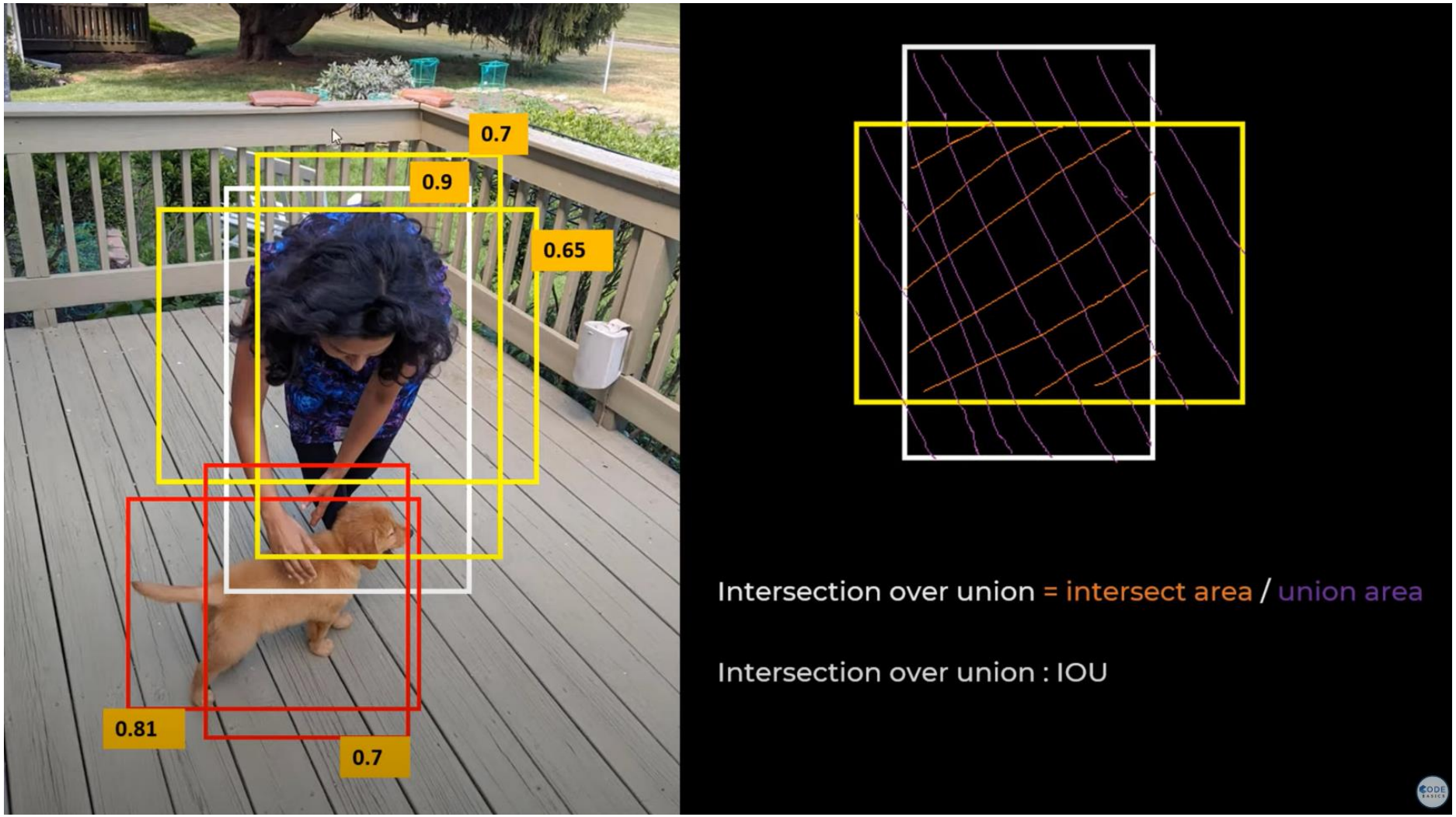

Convolutional Neural Network
16 such vectors
P_c
B_x
B_y
B_w
B_h
C_1
C_2

YOLO (cont'd)

- Let's suppose that those are the final bounding boxes of the model.
- Which one should we keep?



YOLO (cont'd)


0.7
0.9
0.65
0.81
0.7
Intersection over union = intersect area / union area
Intersection over union : IOU

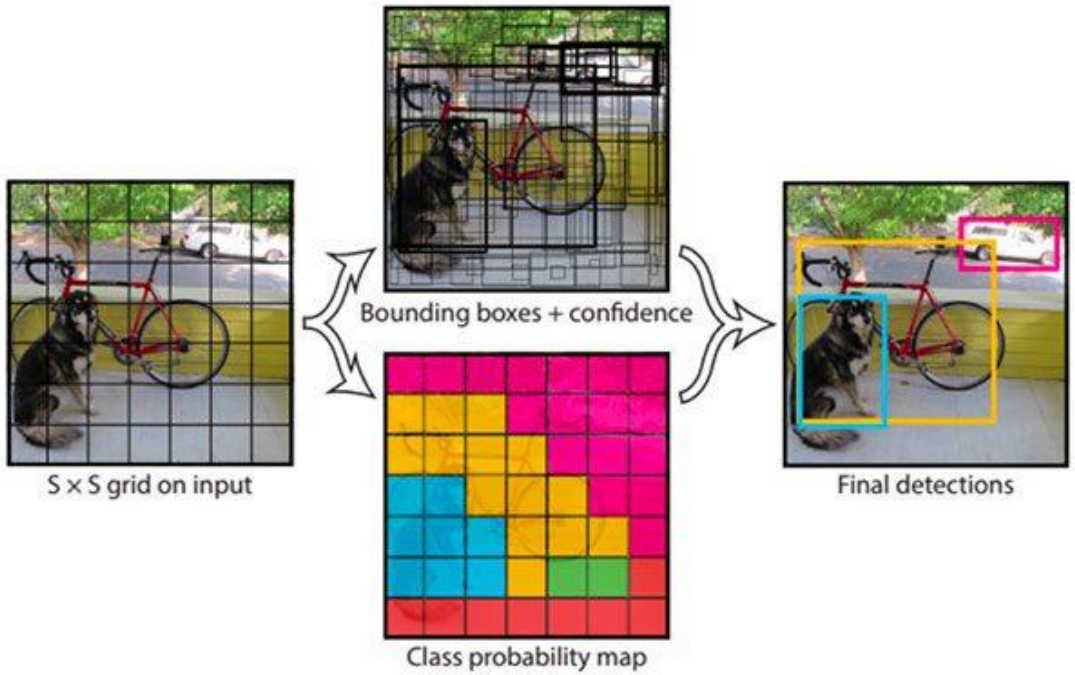
YOLO (cont'd)

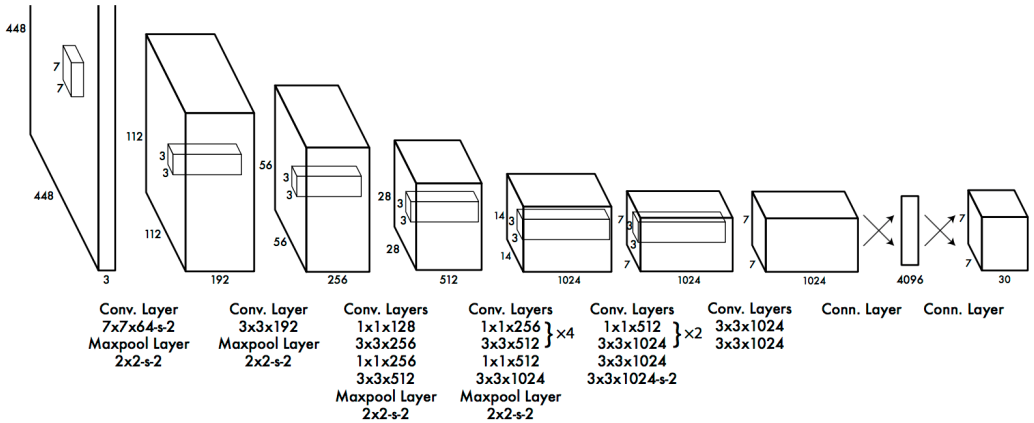
- For each of the bounding boxes, the network outputs a class probability and offset values for the bounding box, including the dimension of the bounding box

- If we have a 4x4 divided images, we will have different proposals. In this case, the intersection of every box is taken as final bounding box

- Intersection over Union (IoU)

YOLO (cont'd)


Bounding boxes + confidence
S × S grid on input
Class probability map
Final detections


448
7
7
448
3
112
3
3
112
192
56
3
3
56
256
28
3
3
28
512
14
3
3
14
1024
7
3
3
7
1024
7
7
1024
4096
7
7
30
Conv. Layer
7x7x64-s-2
Maxpool Layer
2x2-s-2
Conv. Layer
3x3x192
Maxpool Layer
2x2-s-2
Conv. Layers
1x1x128
3x3x256
1x1x256
3x3x512
Maxpool Layer
2x2-s-2
Conv. Layers
1x1x256
3x3x512
×4
1x1x512
3x3x1024
Maxpool Layer
2x2-s-2
Conv. Layers
1x1x512
3x3x1024
×2
3x3x1024
3x3x1024-s-2
Conv. Layers
3x3x1024
3x3x1024
Conn. Layer
Conn. Layer

Exercises